



M2 Deep Learning Coursework

xl628

Department of Physics

8th April 2025

Document Statistics:

Words in text: 2368

Words in headers: 123

Words outside text (captions, etc.): 477

1 Introduction

Recent advances in large language models (LLMs) have opened up new possibilities for recasting time series prediction as a next-token prediction task. Inspired by insights from LLMTime[1], we investigate the use of the `Qwen2.5-0.5B-Instruct` model[3] for forecasting tasks. We transform time series data into discrete tokens through appropriate preprocessing strategies. Furthermore, we employ Low-Rank Adaptation (LoRA)[2] to fine-tune the model, achieving significant improvements, all within tight computational constraints.

2 Data and Preprocessing

The methods presented here are implemented primarily in notebooks `00_data_exploration.ipynb` and `01_llmtime_preprocessing.ipynb`.

2.1 LLMTIME Preprocessing Implementation

Our dataset comprises synthetic time series generated by the Lotka-Volterra predator-prey model, structured as `(1000, 100, 2)`, indicating 1000 distinct simulations, each containing 100 temporal observations of two variables: predator and prey populations.

The paper [1] proposes a method to apply pretrained large language models (LLMs) for continuous time series prediction problems. It represents the time series as a string of numerical digits, and views time series forecasting as next-token prediction in text, unlocking the use of powerful pretrained models and probabilistic capacities, such as likelihood evaluation and sampling.

Scaling and Rounding The Qwen tokenizer inherently decomposes numeric strings into individual digit tokens. For instance, the string "1.23" is tokenized as [16, 13, 17, 18], where 13 is the token ID of the decimal point.

To standardise the numeric range and control token length, we introduce a simple scaling factor α :

$$x'_t = \frac{x_t}{\alpha}$$

where α is chosen based on the empirical data distribution.

Following scaling, we round the resulting values to a fixed number of decimal places to ensure consistency and reduce unnecessary precision.

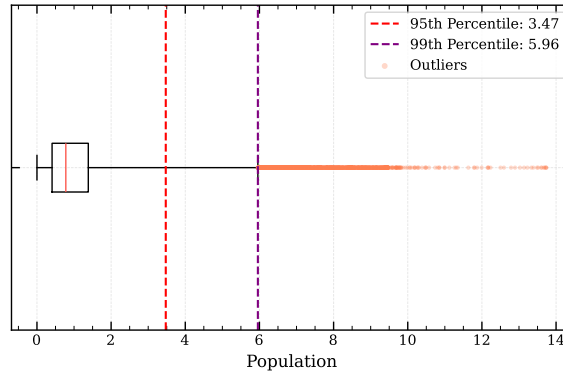


Figure 1: Boxplot of unscaled population values from the entire dataset. The upper and lower edges of the box correspond to the 75th and 25th percentiles, respectively, while the red line in the box denotes the median. Most data falls within $[0, 2]$, and the 99th percentile is approximately 5.96. Coral-coloured points indicate outliers beyond the 99th percentile.

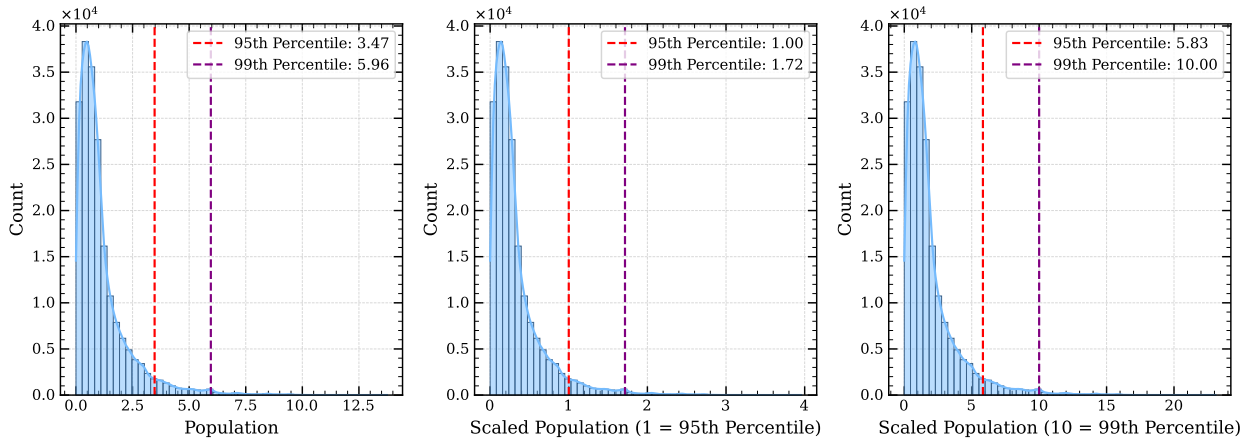


Figure 2: From left to right: (i) unscaled population distribution, (ii) scaled with the 95th percentile mapped to 1, (iii) scaled with the 99th percentile mapped to 10. The 95th-percentile scaling results in most values being compressed into $[0, 1]$, thereby requiring greater decimal precision to retain information.

As seen in Figures 1 and 2, the original data have been primarily distributed within the range $[0, 10]$, meaning the integer part typically consumes only one token. Further compression is therefore unnecessary and may result in information loss. In particular, Figure 3(a) illustrates how mapping

the 95th percentile to 1 introduces noticeable roughness in the trajectories due to limited resolution in low-population regimes.

Although retaining more decimal places improves numeric precision, it incurs a higher token cost. For instance, maintaining three decimal places requires at least five tokens for every value ("`1.234`" $\rightarrow$ [`16`, `13`, `17`, `18`, `19`]).

To balance token efficiency and representational fidelity, we propose **scaling the 99th percentile to 10** and rounding to **two decimal places**. This transformation effectively preserves information while ensuring that roughly **99% of values** require **only four tokens**. Figure 3(b) confirms that this strategy yields smoother and more faithful representations of population trajectories.

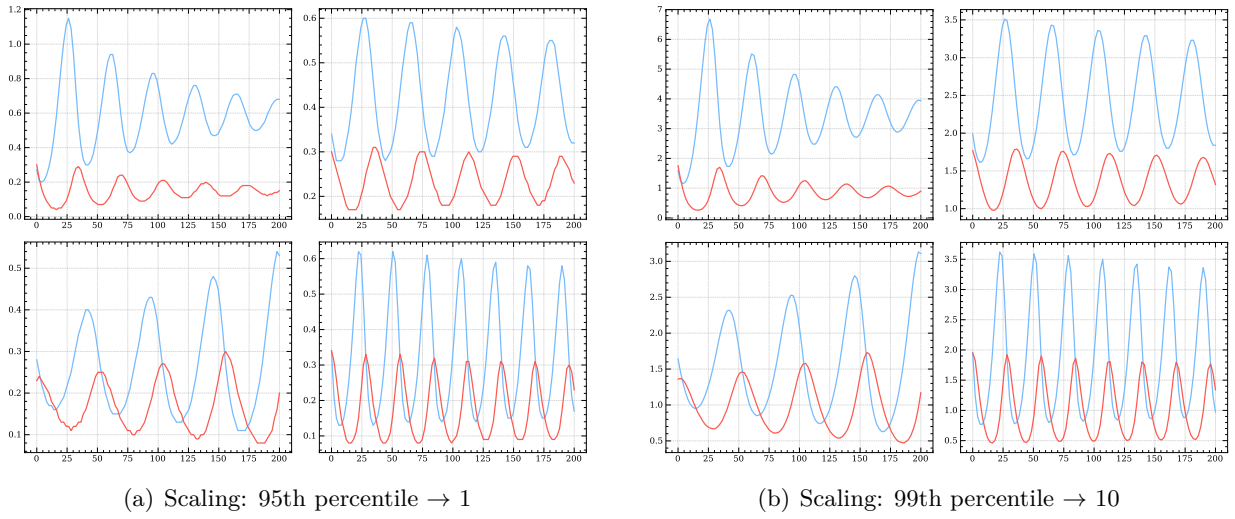


Figure 3: Effect of different scaling strategies on population trajectories across time points. In both cases, data were scaled based on the specified percentile of the dataset, then rounded to two decimal places. (a) shows visibly wiggly and rough behaviour in certain regions, likely due to information loss in low-population regimes. (b) exhibits smoother profiles, suggesting improved resolution preservation.

Encoding Multivariate Data For encoding multivariate time series, we use the following convention:

- Variables at the same timestep are separated by commas ("`,`").
- Different timesteps are separated by semicolons ("`;`").

2.2 Tokenization Example

Let's walk through the process from raw data to tokenized sequences. Refer to Section 5 of the notebook `01_llmtime_preprocessing.ipynb`, where the full pipeline is demonstrated using our implemented preprocessor. An excerpt of the output is below:

Listing 1: Examples showing the full processing pipeline applied to raw population trajectories (slices of 5 time steps). Each example includes the raw input, scaled and rounded version (using a scaling factor of 0.6007—corresponding to the 99th percentile of the training data—and two decimal places), the formatted string representation, and the corresponding tokenized sequence with integer token IDs.

```

--- Example 1 ---
Raw trajectory shape: (5, 2)
Raw trajectory:
[[0.94991744 1.040624  ]
 [0.74055135 0.7795419 ]
 [0.6822457  0.56439036]
 [0.7166742  0.40764433]
 [0.82451135 0.30028325]]

Scaled trajectory:
[[1.58 1.73]
 [1.23 1.3 ]
 [1.14 0.94]
 [1.19 0.68]
 [1.37 0.5 ]]

Formatted text:
1.58,1.73;1.23,1.30;1.14,0.94;1.19,0.68;1.37,0.50

Tokenized sequence (length 49):
[16, 13, 20, 23, 11, 16, 13, 22, 18, 26, 16, 13, 17, 18, 11, 16, 13, 18, 15, 26, 16,
  ↪ 13, 16, 19, 11, 15, 13, 24, 19, 26, 16, 13, 16, 24, 11, 15, 13, 21, 23, 26,
  ↪ 16, 13, 18, 22, 11, 15, 13, 20, 15]

...

--- Example 11 ---
Raw trajectory shape: (5, 2)
Raw trajectory:
[[0.96647114 1.094717  ]
 [0.75811523 0.94788396]
 [0.66393834 0.7895862 ]
 [0.6488795  0.6488567 ]
 [0.69528633 0.5355721 ]]

Scaled trajectory:
[[1.61 1.82]
 [1.26 1.58]
 [1.11 1.31]
 [1.08 1.08]
 [1.16 0.89]]

Formatted text:
1.61,1.82;1.26,1.58;1.11,1.31;1.08,1.08;1.16,0.89

Tokenized sequence (length 49):
[16, 13, 21, 16, 11, 16, 13, 23, 17, 26, 16, 13, 17, 21, 11, 16, 13, 20, 23, 26, 16,
  ↪ 13, 16, 16, 11, 16, 13, 18, 16, 26, 16, 13, 15, 23, 11, 16, 13, 15, 23, 26,
  ↪ 16, 13, 16, 21, 11, 15, 13, 23, 24]

```

3 Baseline Evaluation

In this section, we evaluate the forecasting capabilities of the untrained **Qwen2.5-0.5B-Instruct** model on a tokenized test set comprising 200 samples. The dataset was created via the notebook

01_llmtime_preprocessing.ipynb.

3.1 Untrained Qwen2.5-Instruct Performance

Time series forecasting with pure time series methods typically prioritises recent data over distant past or future projections. Extending too far back can amplify the impact of non-stationarity, where older data ceases to represent current patterns due to shifts in the underlying process. Similarly, forecasting too far ahead compounds uncertainty, leading to larger prediction errors.

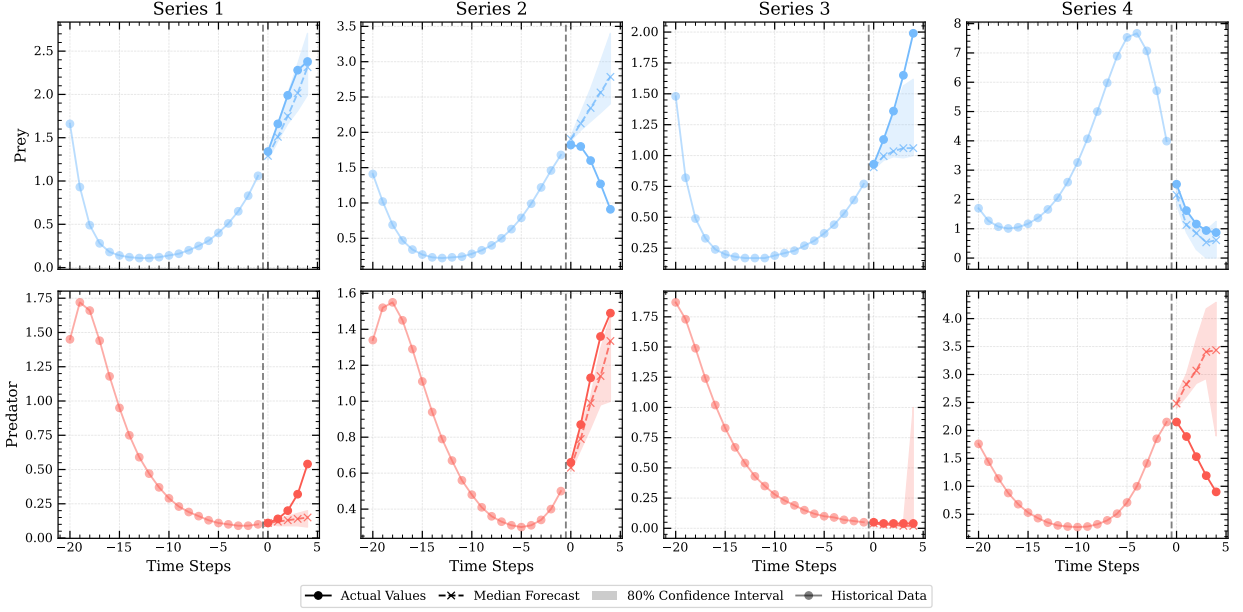


Figure 4: Forecasts generated by the **untrained Qwen2.5-0.5B-Instruct** model on four test samples. Cross markers denote point predictions, while shaded regions represent estimated 80% confidence intervals from Monte Carlo sampling (context steps = 20, forecast steps = 5, temperature = 0.7, top-k = 20).

Given these constraints, we employ relatively short context (20 steps) and forecast lengths (5 steps) in our evaluation. We utilise Monte Carlo sampling, generating 20 samples per forecast from the model’s probabilistic distribution, allowing us to estimate an approximate 80% confidence interval. Figure 4 illustrates four forecasts produced by the untrained model.

Since the model lacks specific training for this task, it frequently produces non-numeric or malformed outputs. We address these issues post hoc, primarily by imputing the last observed valid value in cases of such invalid outputs.

We evaluate the model’s predictions using standard metrics, presented in Table 2, with comparisons to a naive baseline that repeats the last observed value for each variable.

The results demonstrates that the untrained **Qwen2.5-0.5B-Instruct** model consistently outperforms the naive baseline across all the metrics. This suggests that, despite the absence of explicit task-specific training, the model captures some underlying structure from tokenized numeric sequences, benefiting from its pretrained knowledge.

Additionally, effective uncertainty quantification is critical for assessing forecast reliability, especially in financial contexts where risk management heavily relies on understanding predictive uncertainty.

We measure uncertainty quantification through the empirical coverage metric, which measures the proportion of true values falling within the model’s 80% confidence intervals, derived from Monte

Carlo sampling. Ideally, this should be close to 0.8 (80%). However, our untrained model’s coverage (0.3325) is notably lower than desired, suggesting the generated confidence intervals are too narrow or misaligned with the true data distribution.

Adjusting sampling parameters such as temperature or top-k/top-p might mitigate overly deterministic predictions, yet improvements to the model’s core forecasting capability through fine-tuning would likely yield greater overall reliability improvements.

4 LoRA Fine-tuning

In this section, we employ Low-Rank Adaptation (LoRA)[2] to fine-tune the **Qwen2.5-0.5B-Instruct** model specifically for our time series forecasting task.

LoRA inserts trainable low-rank adapters into existing weights without modifying the original parameters. Each LoRA-adapted module performs:

$$Y = XW_0 + X\Delta W = XW_0 + XAB \cdot \frac{\alpha}{r} \quad (1)$$

where $A \in \mathbb{R}^{r \times \text{in_dim}}$ and $B \in \mathbb{R}^{\text{out_dim} \times r}$ are low-rank matrices with rank $r \ll \min(\text{in_dim}, \text{out_dim})$. Here, r controls the rank of the adaptation, and α is a scaling factor set equal to r in our implementation.

LoRA factorizes the parameter update $\Delta W \in \mathbb{R}^{\text{out_dim} \times \text{in_dim}}$ into two smaller matrices (A, B), significantly reducing the number of trainable parameters from $\text{in_dim} \times \text{out_dim}$ to $r \times (\text{in_dim} + \text{out_dim})$.

In our implementation, LoRA is specifically applied to the query and value projection matrices (`q_proj`, `v_proj`) within each transformer layer’s self-attention mechanism. Additionally, we leave the final layer’s bias parameters unfrozen to bias the model towards numerical tokens in our dataset.

See `tnotebook 04_lora_implementation.ipynb` for detailed implementation and a tentative preliminary training result.

Table 1: Hyperparameter settings for initial LoRA fine-tuning of **Qwen2.5-0.5B-Instruct**. Context length represents the maximum token sequence processed, while stride indicates the sliding window step size for processing overlapping data segments.

Hyperparameter	Values
<i>Training Parameters</i>	
Batch Size	8
Maximum Steps	1000
Learning Rate	1×10^{-5}
Evaluation Steps	50
Optimizer	AdamW
<i>LoRA Configuration</i>	
LoRA Rank (r)	4
LoRA α	4 (same as rank)
Target Modules	["q_proj", "v_proj"]
<i>Data Parameters</i>	
Context Length	512
Stride	256

4.1 Initial Experiment

We initially train the 0.5B parameter model with LoRA configurations based on the default hyper-parameters in Table 1 on an NVIDIA A100 GPU (40GB).

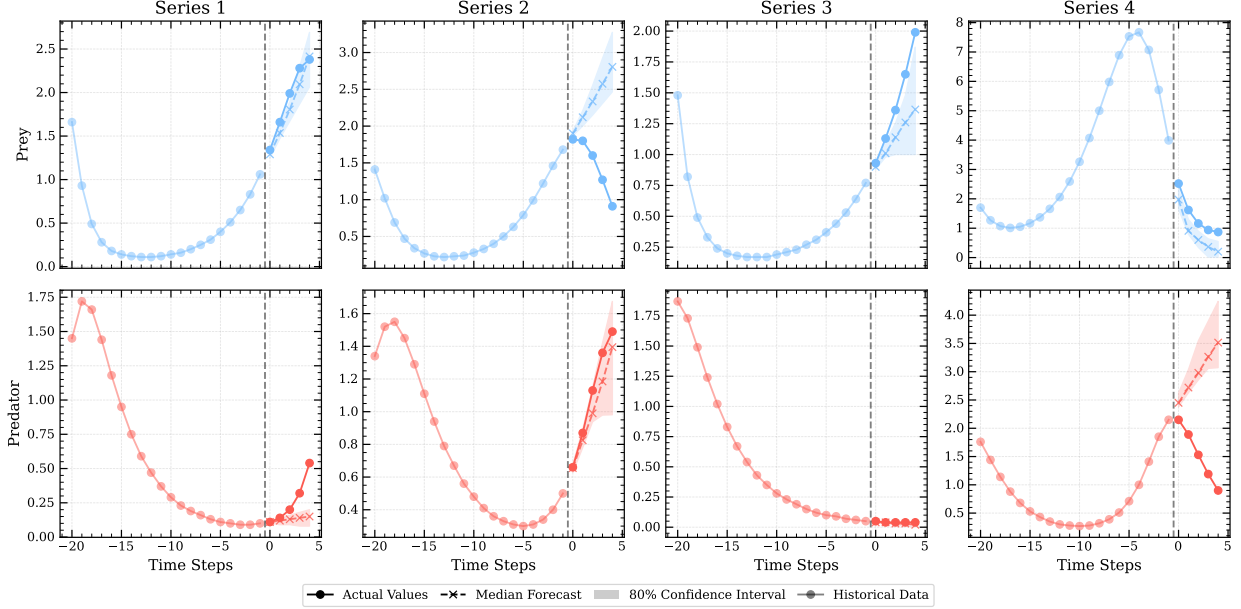


Figure 5: Forecasts produced by the **initial LoRA fine-tuning Qwen2.5-0.5B-Instruct** model on four test samples. Same context and forecast length and sampling parameters as in Figure 4.

Training and evaluation loss curves (cross-entropy) are presented in Figure 6. Both training and evaluation losses steadily decrease, indicating effective learning without noticeable overfitting.

After 1000 steps training with LoRA fine-tuning, the model consistently improves all error metrics, presented in Table 2. These improvements demonstrate that LoRA gradually adapts the model to better fit our forecasting task.

The small increase in coverage and decrease in interval width is also a step forward, showing that LoRA enhances the model’s ability to quantify uncertainty.

These improvements are meaningful but not dramatic. However, they suggest that further training and fine-tuning—such as hyperparameter search—could yield better outcomes. This will be the focus of our next steps.

Table 2: Model Performance Comparison

Metric	Naive Predict	Untrained Baseline	Initial LoRA	Final LoRA
MSE	1.4202	0.8832	0.8069	0.0579
RMSE	0.8451	0.6217	0.5903	0.1289
MAE	0.7612	0.5244	0.4947	0.1049
MAPE (%)	55.73	36.23	34.90	6.53
SMAPE (%)	42.69	31.62	29.18	6.20
Coverage	—	0.3325	0.3520	0.5680
Interval Width	—	0.5100	0.5050	0.1661

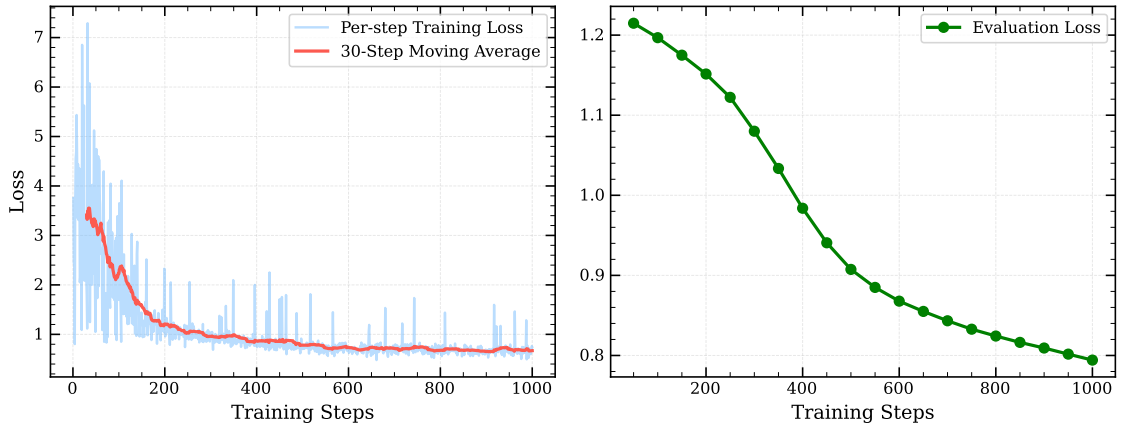


Figure 6: Initial training loss curves. Left: Per-step training loss (light blue) with a 30-step moving average (red) shows a consistent downward trend. Right: Evaluation loss (green) also steadily declines.

Table 3: Hyperparameter configurations used for LoRA fine-tuning of Qwen2.5-Instruct.

Hyperparameter	Values
<i>Hyperparameters Varied During Search</i>	
Learning Rate	$\{1 \times 10^{-5}, 5 \times 10^{-5}, 1 \times 10^{-4}\}$
LoRA Rank (r)	$\{2, 4, 8\}$
Context Length	$\{128, 512, 768\}$
<i>Fixed Hyperparameters</i>	
Batch Size	8
Maximum Steps (Main)	250
Maximum Steps (Context)	200
Evaluation Frequency	25
Optimizer	AdamW
LoRA α	Same as rank
Target Modules	["q_proj", "v_proj"]
Stride	256

4.2 Hyperparameter Tuning

We performed hyperparameter tuning through grid search, adjusting learning rates, LoRA ranks, and context lengths according to configurations shown in Table 3. The tuning process consisted of two stages: an initial exploration of learning rates and LoRA ranks, followed by context length optimisation based on the best-performing parameters identified initially.

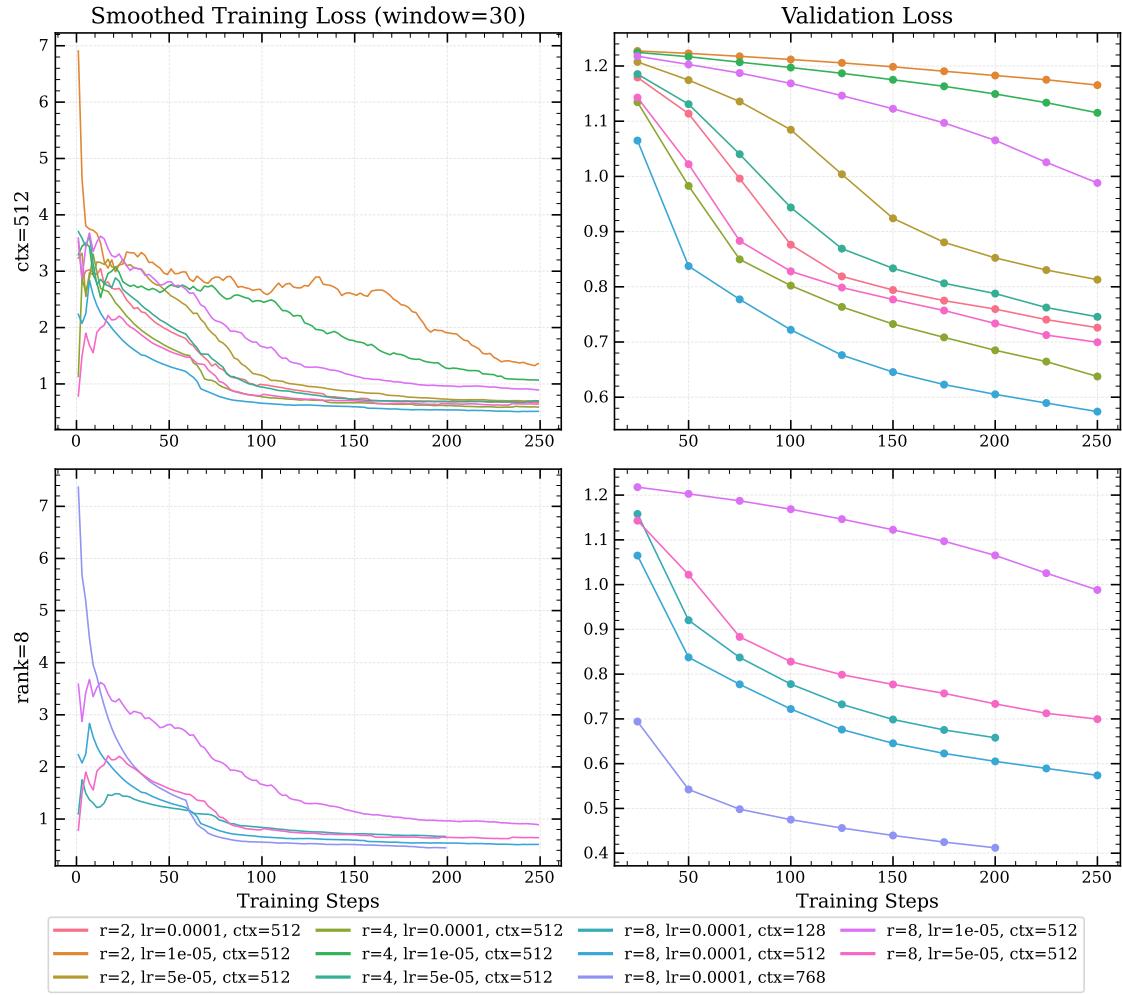


Figure 7: Comparison of smoothed training loss (left column) and validation loss (right column) across different LoRA ranks, learning rates, and context lengths. Top row: Results for context length 512 across multiple ranks ($r = 2, 4, 8$) and learning rates ($1e-5, 5e-5, 1e-4$). Bottom row: Fixed the best parameters from the previous search — rank $r = 8$ and learning rate $lr = 0.0001$ — and varied context lengths (128, 512, 768), while also including the other two learning rates at context length 512 for comparison.

The training and validation losses of all parameter combinations are showed in Figure 7, showing **higher LoRA ranks, larger learning rates, and longer context lengths** yield faster convergence and improved model performance.

Figure 8 summarises the optimal validation loss across different combinations of LoRA rank and learning rate. Again, the trend shows that increasing both the LoRA rank and the learning rate leads to improved model performance, as evidenced by decreasing validation loss.

The results indicate that the best configuration in this search is `learning_rate=1e-4`, `lora_rank=8`. We further explored context lengths of `[128, 512, 768]`, finding that the longest context length `ctx_length=768` yielded the best performance. Thus, these hyperparameters were selected for the final training phase.

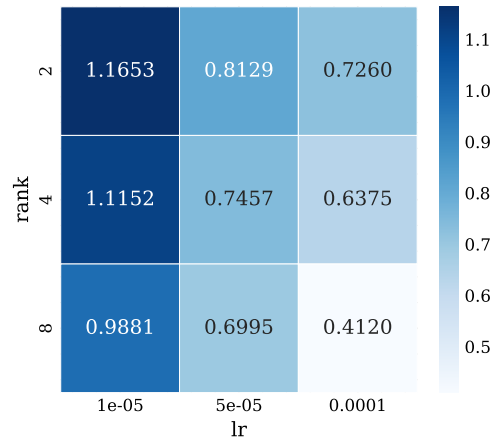


Figure 8: Heatmap of best validation loss across different combinations of LoRA rank and learning rate (lr). Lighter shades of blue indicate lower loss values, reflecting better model performance.

4.3 Final Model Training

To manage the FLOPS budget, we limited the final training to 2,000 steps, although training curves suggest that additional training may yield further improvements.

An important observation from this final training is the absence of previously encountered non-numeric or unstructured outputs, as evidenced by the logging files. This indicates that the model has effectively learned the structure of the numeric time series tokens. In other words, the fine-tuned model demonstrates strong task-specific adaptation.

The final LoRA model achieves extraordinary improvements in all evaluated metrics compared to prior models, as detailed in Table 2. For instance, the mean absolute error (MAE) dramatically drops from 0.7612 (naive), 0.5244 (untrained) and 0.4947 (initial LoRA) down to 0.1049 (final LoRA).

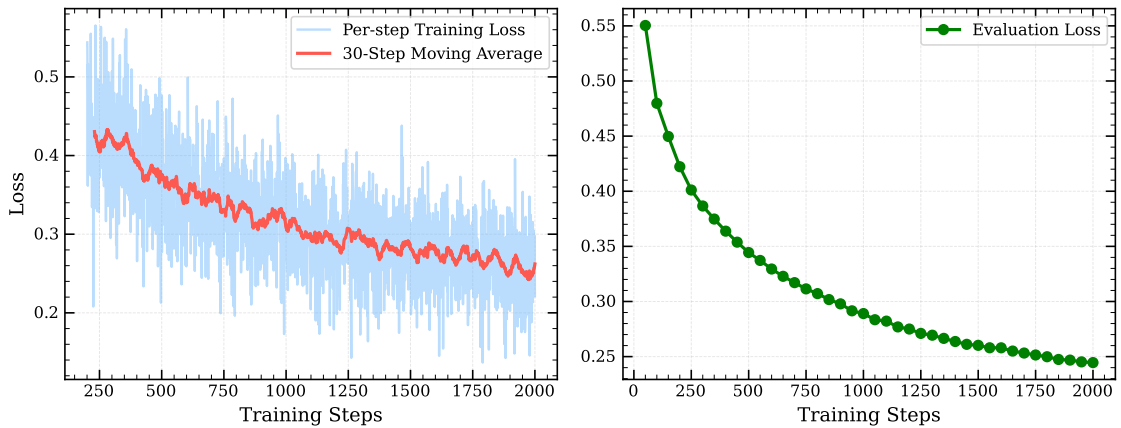


Figure 9: Final training of the best-performing model. Left: Per-step training loss with a 30-step moving average, shown from step 200 onward, continues to decrease. Right: Evaluation loss maintains a consistent downward trajectory.

Coverage increases to 0.5680, a significant leap from 0.3325 (untrained) and 0.3520 (initial LoRA), with a much narrower interval width (0.1661 vs. 0.5100 and 0.5050). This combination indicates that the final LoRA model not only predicts more accurately but also provides tighter,

better-calibrated confidence intervals. However, since coverage remains below the desired 0.8, there is room for additional refinement in uncertainty quantification.

Although performance substantially improved, the continual downward trends observed in both training and evaluation losses (Figure 9) suggest the potential for even greater performance with extended training. The incomplete hyperparameter search also implies that the model may not yet be at its peak performance.

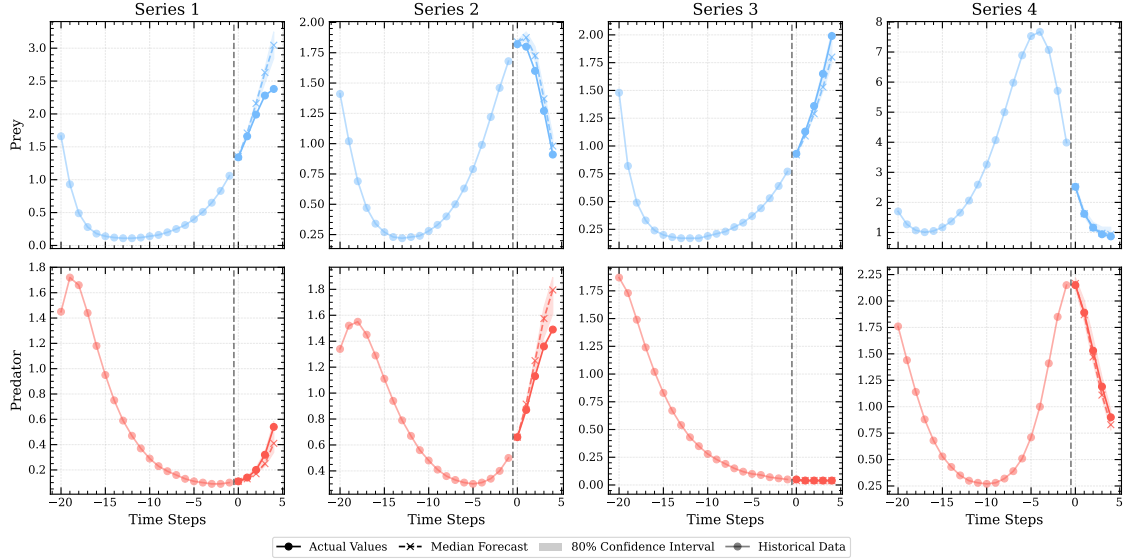


Figure 10: Forecasts generated by the **Final LoRA fine-tuned Qwen2.5-0.5B-Instruct** model on four test samples, using the same context, forecast length, and sampling parameters as Figure 4. The prediction trajectories show significant improvement over those in Figures 5 and 4.

Forecast visualizations in Figure 10 illustrate clear improvements compared to earlier model versions (Figures 5 and 4), reinforcing the significant advancements achieved through the final LoRA fine-tuning.

5 FLOPS Calculation for Qwen2.5-Instruct Models

The notebook `03_flops_calculation.ipynb` provides examples of FLOPs calculation specified to Qwen2.5-0.5B hyperparameters. A concise yet comprehensive summary of the computational breakdown is presented in Table 6.

Notation & FLOPS Accounting Table

In this analysis, we account for basic operations as in Table 5.

Matrix multiplication $m \times n$ by $n \times p$ (without bias) is $m \times p \times (2n - 1)$ FLOPS (n multiplications, $n - 1$ additions per element). With bias, it's $m \times p \times 2n$ (adds n additions for bias).

This represents a simplified accounting method. Other primitives that cannot be expressed with these are assumed to be 0 FLOPS, such as array indexing, memory management, etc.

Symbol	Description
B	Batch size (number of sequences per forward pass)
S	Sequence length (number of tokens per sequence)
H	Hidden size (embedding dimension)
I	Intermediate size in the MLP block
V	Vocabulary size for the language modeling head
L	Number of transformer layers in the model
N_h	Number of attention heads
D_h	Dimension per head ($D_h = H/N_h$)
r	LoRA rank (dimensionality of low-rank adapters)
M	Number of LoRA-injected modules (e.g., $2L$ for Q and V projections)
T	Number of training steps or optimization iterations
$\mathcal{F}$	FLOPs (floating-point operations) for a given module or total

Table 4: Notation used in FLOPs and LoRA computation.

Operation	FLOPS
Addition/Subtraction/Negation (float or integer)	1
Multiplication/Division/Inverse (float or integer)	1
ReLU/Absolute Value	1
Exponentiation/Logarithm	10
Sine/Cosine/Square Root	10

Table 5: FLOPS accounting for primitive operations.

5.1 Detailed FLOPS Analysis

5.1.1 Input Processing

Token Embeddings After tokenization, each input token is mapped to an H -dimensional vector via an embedding lookup. Conceptually, this corresponds to a matrix multiplication of shape $B \times S \times V @ V \times H$ requiring $B \times S \times H \times (2V - 1)$ FLOPS. However, in practice, this operation is implemented as a lookup table and conventionally considered to require **zero FLOPS**.

Positional Encodings Qwen2.5 employs **Rotary Position Embeddings (RoPE)** [4] to encode positional information within the attention mechanism through structured rotations of query and key vectors.

For simplicity, we approximate this by assuming the addition of precomputed positional encodings to token embeddings. With embeddings of shape $B \times S \times H$ and positional encodings of shape $S \times H$ (broadcasted across the batch dimension), this requires $B \times S \times H$ FLOPS for element-wise addition.

5.1.2 Transformer Layers

Each of the L identical transformer layers processes the sequence as follows:

Root Mean Square Layer Normalization (RMSNorm) Qwen2.5 applies RMSNorm [5] to the input tensor $X \in \mathbb{R}^{B \times S \times H}$ according to:

$$\tilde{x}_i = \frac{x_i}{\sqrt{\frac{1}{H} \sum_{j=1}^H x_j^2 + \epsilon}} \cdot g_i$$

FLOPS breakdown per token (H -dimensional vector):

- H multiplications for squaring each element
- $(H - 1)$ additions to sum the squared elements
- 1 multiplication by $\frac{1}{H}$ and 1 addition of ϵ
- 1 square root operation (10 FLOPS)
- H divisions and H multiplications by scale factors g_i

Total per token: $3H + 1 + 10 + H = 4H + 11$ FLOPS

For the entire batch: $B \times S \times (4H + 11)$ FLOPS

Self-Attention Mechanism Although Qwen2.5 uses Grouped Query Attention (GQA), we approximate it as standard multi-head attention for simplicity.

(a) Query, Key, Value Projections

$$Q = XW_Q + b_Q, \quad K = XW_K + b_K, \quad V = XW_V + b_V \quad (2)$$

The input $X \in \mathbb{R}^{B \times S \times H}$ is projected via three linear transformations to obtain query, key, and value representations. Each projection requires a matrix multiplication of shape $(B \times S) \times H \times H$ plus bias addition, totaling $B \times S \times 2H^2$ FLOPS per projection.

For all three projections: $3 \times B \times S \times 2H^2 = 6BSH^2$ FLOPS

(b) Multi-Head Attention The H -dimensional embeddings are split into N_h heads, each with dimension $D_h = H/N_h$.

(c) Attention Scores and Weights

$$A_h = \text{softmax} \left(\frac{Q_h K_h^\top}{\sqrt{D_h}} \right) \quad (3)$$

Computing attention weights requires:

- Matrix multiplication $Q_h K_h^\top$ for each head: $N_h \times B \times S \times S \times (2D_h - 1)$ FLOPS
- Scaling by $1/\sqrt{D_h}$: $N_h \times B \times S \times S$ FLOPS
- Softmax normalization, including exponentiation (10 FLOPS per element), summation, and division: $N_h \times B \times S \times (12S - 1)$ FLOPS

(d) Context Aggregation

$$\text{Output}_h = A_h V_h \in \mathbb{R}^{B \times S \times D_h} \quad (4)$$

Multiplying attention weights with values: $N_h \times B \times S \times D_h \times (2S - 1)$ FLOPS

(e) Output Projection The concatenated attention outputs from all heads are projected back to the hidden dimension via $W_O \in \mathbb{R}^{H \times H}$, requiring $B \times S \times H \times (2H - 1)$ FLOPS.

Residual Connection The attention output is added to the input, requiring $B \times S \times H$ FLOPS.

Feed-Forward Network Qwen2.5 uses a SwiGLU-activated feed-forward network:

$$\text{Output} = [(XW_{\text{up}}) \odot \text{SiLU}(XW_{\text{gate}})] W_{\text{down}} \quad (5)$$

where $\text{SiLU}(x) = x \odot \sigma(x)$ (sigmoid multiplication).

FLOPS breakdown:

- Gate projection (XW_{gate}): $B \times S \times I \times (2H - 1)$ FLOPS
- Up projection (XW_{up}): $B \times S \times I \times (2H - 1)$ FLOPS
- SiLU activation, including sigmoid (approximately 13 FLOPS per element) and multiplication: $B \times S \times I \times 14$ FLOPS
- Element-wise multiplication between gate and up projections: $B \times S \times I$ FLOPS
- Down projection: $B \times S \times H \times (2I - 1)$ FLOPS

Second Residual Connection Adding FFN output to the normalized attention output: $B \times S \times H$ FLOPS.

5.1.3 Output Processing

Final RMSNorm Identical to previous RMSNorm layers: $B \times S \times (4H + 11)$ FLOPS.

Language Modeling Head Projects normalized embeddings to vocabulary logits via $W_{LM} \in \mathbb{R}^{H \times V}$, requiring $B \times S \times V \times 2H$ FLOPS (with bias). Typically, W_{LM} shares weights with the input embedding matrix.

5.1.4 LoRA Modules

FLOPS per LoRA Module

$$\mathcal{F}_{\text{LoRA, module}} = B \cdot S \cdot (H \cdot (2r - 1) + r \cdot (2H - 1) + H) \quad (6)$$

Simplifying: $\mathcal{F}_{\text{LoRA, module}} = B \cdot S \cdot (4rH - r)$

This includes:

- Down-projection (XA): $B \cdot S \cdot r \cdot (2H - 1)$ FLOPS
- Up-projection (XAB): $B \cdot S \cdot H \cdot (2r - 1)$ FLOPS
- Addition to original output: $B \cdot S \cdot H$ FLOPS

Total LoRA Overhead For M LoRA-injected modules:

$$\mathcal{F}_{\text{LoRA, total}} = M \cdot B \cdot S \cdot (4rH - r) \quad (7)$$

5.2 Total Computation

Forward Pass The total FLOPS for a single forward pass is the sum of all component FLOPS multiplied by the number of layers where applicable.

Training Step For training, the backward pass approximately doubles the forward pass FLOPS:

$$\mathcal{F}_{\text{training step}} = 3 \cdot \mathcal{F}_{\text{forward pass}} \quad (8)$$

Inference Because we employ the validation loss to select the optimal model checkpoint, the associated computational cost is included in the overall training overhead.

Each inference corresponds to a single forward pass; thus, we define:

$$\mathcal{F}_{\text{inference}} = \mathcal{F}_{\text{forward pass}} \quad (9)$$

Cumulative Training FLOPS For T_{train} training steps and T_{val} validation steps:

$$\mathcal{F}_{\text{total}} = T_{\text{train}} \cdot \mathcal{F}_{\text{training step}} + T_{\text{val}} \cdot \mathcal{F}_{\text{inference}} \quad (10)$$

Component	Input Shape	Output Shape	FLOPs	Formula/Notes
I. Initial Processing				
Token Embedding	$B \times S$	$B \times S \times H$	0	Lookup operation
Positional Encoding	$B \times S \times H$	$B \times S \times H$	$B \times S \times H$	Element-wise addition
II. Transformer Layers ($\times 24$)				
Input RMSNorm	$B \times S \times H$	$B \times S \times H$	$B \times S \times (4H + 11)$	$\tilde{x}_i = \frac{x_i}{\sqrt{\frac{1}{H} \sum_{j=1}^H x_j^2 + \epsilon}} \cdot g_i$
Q/K/V Projections	$B \times S \times H$	$B \times S \times H$ (each)	$6 \times B \times S \times H^2$	Three matrix multiplications with bias
Attention Computation:				
Attention Scores	$B \times N_h \times S \times D_h$	$B \times N_h \times S \times S$	$N_h \times B \times S^2 \times (2D_h - 1)$	QK^T where $D_h = H/N_h$
Scaling	$B \times N_h \times S \times S$	$B \times N_h \times S \times S$	$N_h \times B \times S^2$	Division by $\sqrt{D_h}$
Softmax	$B \times N_h \times S \times S$	$B \times N_h \times S \times S$	$N_h \times B \times S \times (12S - 1)$	$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$
Weighted Sum	$B \times N_h \times S \times S$	$B \times N_h \times S \times D_h$	$N_h \times B \times S \times D_h \times (2S - 1)$	$A \times V$
Output Projection	$B \times S \times H$	$B \times S \times H$	$B \times S \times H \times (2H - 1)$	Matrix multiplication
Residual Connection (1)	$B \times S \times H$	$B \times S \times H$	$B \times S \times H$	Element-wise addition
Post-Attn RMSNorm	$B \times S \times H$	$B \times S \times H$	$B \times S \times (4H + 11)$	Same as Input RMSNorm
Feed-Forward Network:				
Gate Projection	$B \times S \times H$	$B \times S \times I$	$B \times S \times I \times (2H - 1)$	Matrix multiplication
Up Projection	$B \times S \times H$	$B \times S \times I$	$B \times S \times I \times (2H - 1)$	Matrix multiplication
SwiGLU Activation	$B \times S \times I$	$B \times S \times I$	$B \times S \times I \times 14$	SiLU: $x \cdot \sigma(x)$
Down Projection	$B \times S \times I$	$B \times S \times H$	$B \times S \times H \times (2I - 1)$	Matrix multiplication
Residual Connection (2)	$B \times S \times H$	$B \times S \times H$	$B \times S \times H$	Element-wise addition
III. Final Processing				
Final RMSNorm	$B \times S \times H$	$B \times S \times H$	$B \times S \times (4H + 11)$	Same as previous RMSNorms
Language Modeling Head	$B \times S \times H$	$B \times S \times V$	$B \times S \times V \times 2H$	Matrix multiplication with bias

Table 6: Computation breakdown for transformer architecture with shapes and FLOPs.

5.3 Computational Resource Management

The FLOP usage of each experiment across our work is summarised in Table 7.

Table 7: FLOP Usage Summary Across All Experiments (Total Budget: 1×10^{17} FLOPs)

Experiment	Rank	Ctx	Steps	Batch Size	FLOPS ($\times 10^{15}$)	% Budget
<i>Initial Experiment</i>						
Initial	4	512	1000	8	13.64	13.64%
<i>Hyperparameter Search</i>						
lora_r2_lr1e-05_ctx512	2	512	250	8	3.43	3.43%
lora_r4_lr1e-05_ctx512	4	512	250	8	3.43	3.43%
lora_r8_lr1e-05_ctx512	8	512	250	8	3.43	3.43%
lora_r2_lr5e-05_ctx512	2	512	250	8	3.43	3.43%
lora_r4_lr5e-05_ctx512	4	512	250	8	3.43	3.43%
lora_r8_lr5e-05_ctx512	8	512	250	8	3.43	3.43%
lora_r2_lr0.0001_ctx512	2	512	250	8	3.43	3.43%
lora_r4_lr0.0001_ctx512	4	512	250	8	3.43	3.43%
lora_r8_lr0.0001_ctx512	8	512	250	8	3.43	3.43%
lora_r8_lr0.0001_ctx128	8	128	200	8	0.67	0.67%
lora_r8_lr0.0001_ctx768	8	768	200	8	4.21	4.21%
<i>Final Training</i>						
Final	8	768	2000	8	41.79	41.79%
Total					91.18	91.18%

6 Conclusion

Despite modest training and limited hyperparameter tuning, our experiments demonstrate that LoRA fine-tuning is a viable and efficient approach for adapting large language models to time series forecasting, even at small scales.

In low-resource settings, short but strategic training with careful validation monitoring offers a practical way to explore performance across the hyperparameter space of the approach. This provides valuable insights and lays a solid foundation for scaling future efforts.

Low-Rank Adaptation (LoRA) proves particularly effective in this context. By freezing most model parameters and training only low-rank adapters, we achieve substantial performance gains at minimal computational cost. Additionally, retaining the language model head bias effectively improves output calibration, enhancing the model’s ability to generate targeted tokens.

For next steps, there are several directions to further enhance model performance: **1. Extended hyperparameter tuning:** Further exploration of learning rates, LoRA ranks, context lengths and other hyperparameters. **2. Longer Training Regimes:** While our final model shows promising results after 2,000 training steps, extended training may further refine model predictions and generalisation. **3. Enhanced uncertainty quantification:** Consider using alternative sampling strategies or adjusting LLM parameters like temperature and top- k to improve the calibration of the model’s confidence intervals.

A Use of AI Tools

In this report, AI tools were utilised to enhance the quality and presentation of the content. Their contributions include:

- **Drafting** – Assisted in writing various sections of the report, including the FLOPS Calculation section, Introduction, Conclusion, and Appendix. Also used to generate captions for figures and tables.
- **Proofreading** – Reviewed grammar, improved sentence structure, and suggested alternative wordings to enhance clarity and coherence throughout the report.
- **LaTeX Assistance** – Helped resolve issues encountered during the LaTeX formatting process, such as debugging compilation errors and formatting well-structured tables.

B Template Information

This report was formatted using the LaTeX template provided in the Example Coursework by Michal Dorko, with some modifications applied.

References

- [1] Nate Gruver et al. *Large Language Models Are Zero-Shot Time Series Forecasters*. 2024. arXiv: [2310.07820](https://arxiv.org/abs/2310.07820) [cs.LG]. URL: <https://arxiv.org/abs/2310.07820>.
- [2] Edward J. Hu et al. *LoRA: Low-Rank Adaptation of Large Language Models*. 2021. arXiv: [2106.09685](https://arxiv.org/abs/2106.09685) [cs.CL]. URL: <https://arxiv.org/abs/2106.09685>.
- [3] Qwen et al. *Qwen2.5 Technical Report*. 2025. arXiv: [2412.15115](https://arxiv.org/abs/2412.15115) [cs.CL]. URL: <https://arxiv.org/abs/2412.15115>.
- [4] Jianlin Su et al. *RoFormer: Enhanced Transformer with Rotary Position Embedding*. 2023. arXiv: [2104.09864](https://arxiv.org/abs/2104.09864) [cs.CL]. URL: <https://arxiv.org/abs/2104.09864>.
- [5] Biao Zhang and Rico Sennrich. *Root Mean Square Layer Normalization*. 2019. arXiv: [1910.07467](https://arxiv.org/abs/1910.07467) [cs.LG]. URL: <https://arxiv.org/abs/1910.07467>.